

Perylene - DMA in ACN

I will try to simulate Pe and DMA in ACN in gromacs for 10 ns.

First I calculated the opt+freq of Pe and DMA separately. Using multiwfn I calculated the ChelpG charges. Using Avogadro I exported the .fchk files to .mol2 and using a python script I replaced the charges by the chelpG charges from multiwfn inside the mol2 file.

Using acpype I generate the topology files for the both molecules:

```
acpype -i Pe.mol2 -n 0 -m 1 -c user
```

```
acpype -i DMA.mol2 -n 0 -m 1 -c user
```

the -c user flag specifies to take the charges directly from the user i.e. from the .mol2 file.

Then I copied the .itp and .gro files for both molecules outside their respective acpype folders.

In the Pe.itp I copied/cut the [defaults] from the ACN.itp. I did the same with the [atom types] and added the missing ones from both the DMA.itp and the ACN.itp. **Make sure you don't have duplicate atom types otherwise you'll get an error later**

In all the .gro and .itp files I changed the resnames to = PER, DMA or ACN

Then I made a topol.top file with the following content:

```
!cat MD/topol.top

#include "./Pe.itp"
#include "./DMA.itp"
#include "./ACN.itp"

[ system ]
; Name
Pe in ACN

[ molecules ]
; Compound #mols
Pe          1
DMA         1
ACN        2500
```

Making a Box

Then I made a 5 nm x 5 nm x 5 nm box with Pe in the center using:

```
gmx editconf -f Pe.gro -bt cubic -box 5 5 5 -o box.gro -center 2.5 2.5 2.5
```

Then I added 1 DMA molecule inside the box using:

```
gmx insert-molecules -f box.gro -ci DMA.gro -nmol 1 -o box2.gro -try 10000
```

Then I added all the ACN according to the density and the volume. Here it should be:

```
from VautheyLab.standard import *

# molar mass of ACN / g/mol
M = 41.05

# density / g/mL
rho = 0.7822

# avogadros constant
Na = sc.Avogadro

# legth of the box in / m
L = 5e-9
# volume in m^3
V = L**3
# convert to mL
V = V*10**6

# calculate the required number of ACN molecules
N = (rho*Na*V)/M

print(f"N = {N}")

N = 1434.3844404604142
```

So I used this command again:

```
gmx insert-molecules -f box2.gro -ci ACN.gro -nmol 1434 -o box3.gro -try 10000
```

the finial box looks like:

Also make sure to **edit the topol.top** so that it has the right number of DMA and ACN.

```
!cat MD/topol.top

#include "./Pe.itp"
#include "./DMA.itp"
#include "./ACN.itp"

[ system ]
; Name
```

```
Pe in ACN

[ molecules ]
; Compound #mols
Pe          1
DMA         1
ACN        1434
```

Energy Minimization

I ran first the grompp using:

```
gmx grompp -f min.mdp -c box3.gro -p topol.top -o min/min
```

And then the actual optimization using:

```
gmx mdrun mdrun -v -deffnm min/min
```

and output the potential energy using:

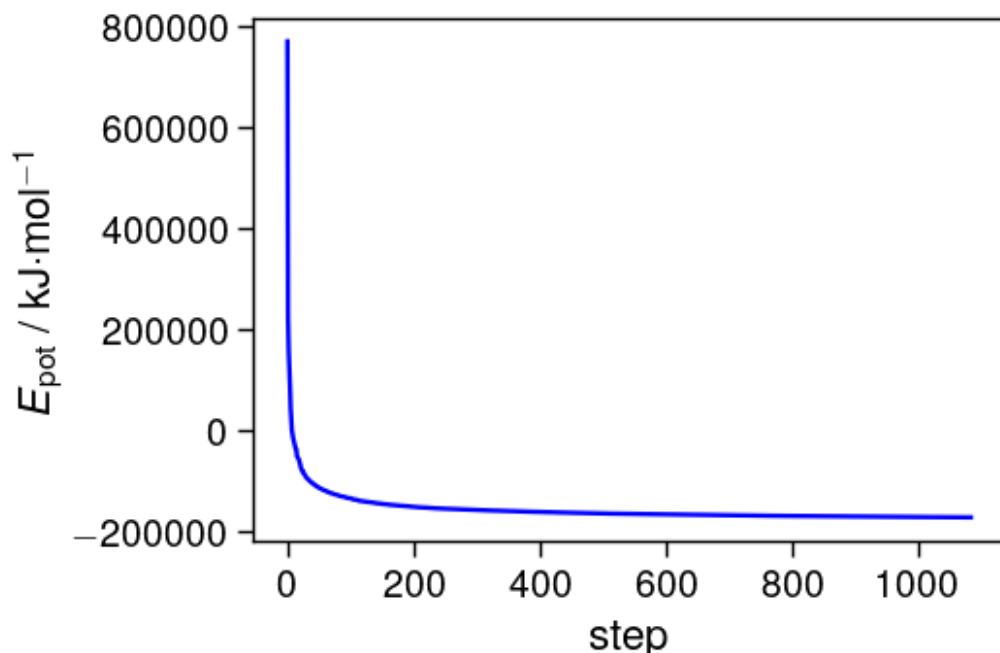
```
gmx energy -f min/min.edr -o min/ener.xvg (10)
```

Let's see if it converged!

```
data = np.loadtxt('MD/min/ener.xvg', skiprows=24)
step = data[:,0]
Epot = data[:,1]

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(step, Epot, '-b')
ax.set_ylabel(r'$E_{\text{pot}}$ / kJ$\cdot\text{mol}^{-1}$')
ax.set_xlabel(r'step')
#ax.set_xscale('log')

Text(0.5, 0, 'step')
```



Equillibration - NVT

For the NVT equillibration I ran as before:

```
gmx grompp -f nvt.mdp -c min/min.gro -p topol.top -o nvt/nvt
```

and then:

```
gmx mdrun -v -deffnm nvt/nvt
```

for 100 ps in steps of 1 fs.

Then I output the temperature as usual:

```
gmx energy -f nvt/nvt.edr -o nvt/temp.xvg (15)
```

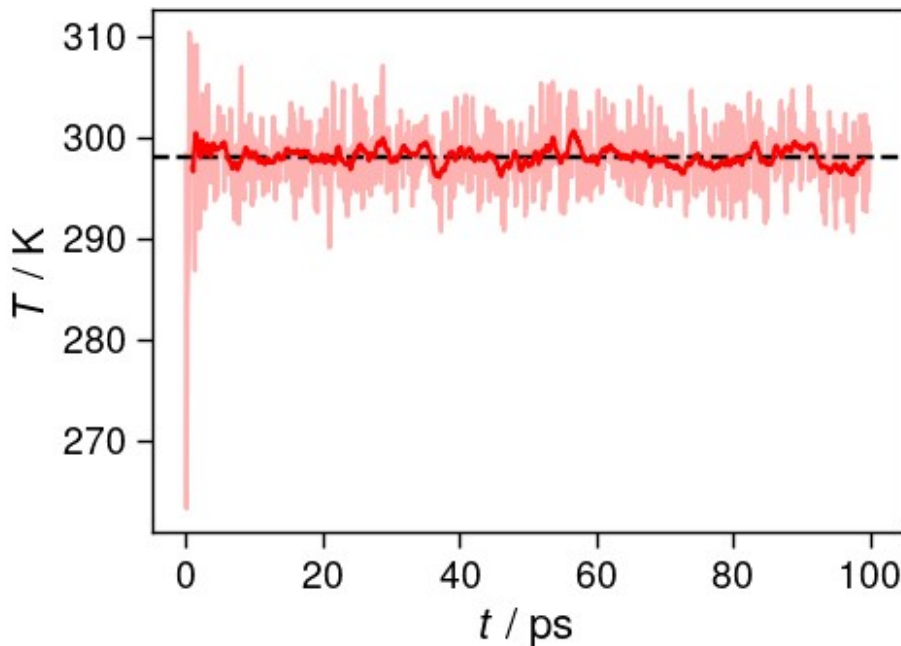
Let's see if the temperature reached its target value!

```
from VautheyLab.miscellaneous import moving_average as ma

data = np.loadtxt('MD/nvt/temp.xvg', skiprows=24)
t = data[:,0]
T = data[:,1]

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(t, T, '-r', alpha=0.3)
ax.axhline(y=298.15, linestyle='--', color='k')
ax.plot(ma(t, 20), ma(T,20), '-r')
ax.set_ylabel(r'$T$ / K')
ax.set_xlabel(r'$t$ / ps')
```

```
Text(0.5, 0, '$t$ / ps')
```



Equilibration - NPT

I first generated the input for the mdrun using grompp as usual:

```
gmx grompp -f npt.mdp -c nvt/nvt.gro -p topol.top -o npt/npt
```

I actually had to run this command **on the cluster as well** to do the subsequent MD run. This is due to the fact that GROMACS 2023 is installed on the server and on my PC I have GROMACS 2024.

Make sure to **replace the compressibility by the value of your solvent inside the npt.mdp**. I will run the NPT-eql. for 1 ns in steps of 1 fs.

The actual MD run I ran on Bamboo using the following input script:

```
!cat MD/JOB_NPT.sh

#!/bin/bash
#SBATCH --job-name=Pe_DMA_NPT
#SBATCH --output=slurm-%J.out
#SBATCH --error=slurm-%J.err
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --time=00-4:00:00
#SBATCH --partition=shared-cpu
```

```
module load GCC/11.3.0 CUDA/11.4.1 OpenMPI/4.1.4 GROMACS/2023.1
gmx mdrun -nt 32 -v -deffnm npt/npt
```

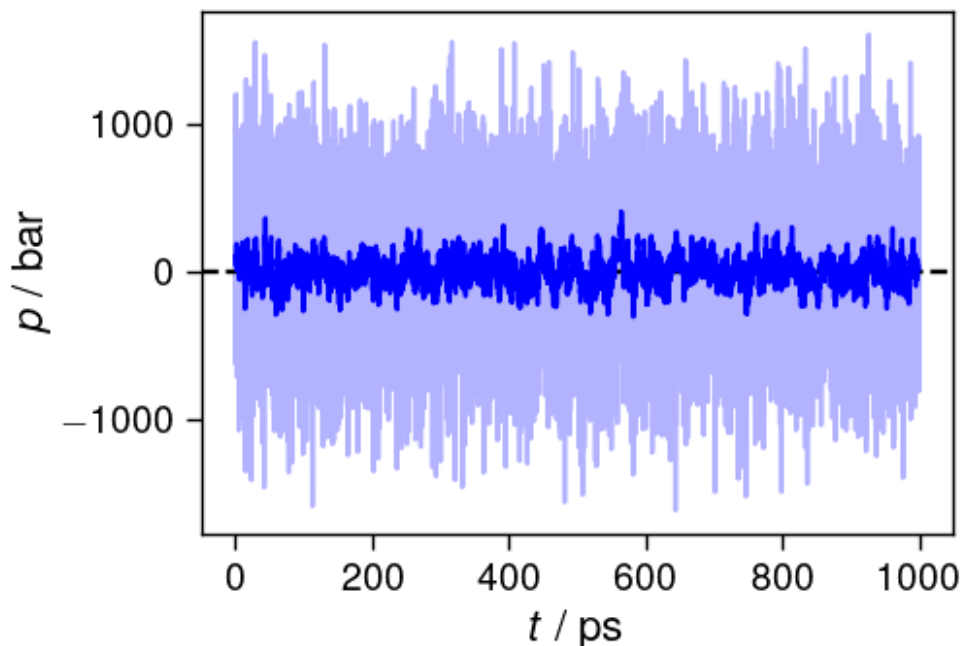
Let's see if the density and pressure converged. I output them again via the usual method, i.e. :

```
gmx energy -f npt/npt.edr -o npt/pressure.xvg (17)
```

```
gmx energy -f npt/npt.edr -o npt/density.xvg (23)
```

```
data = np.loadtxt('MD/npt/pressure.xvg', skiprows=24)
t = data[:,0]
p = data[:,1]

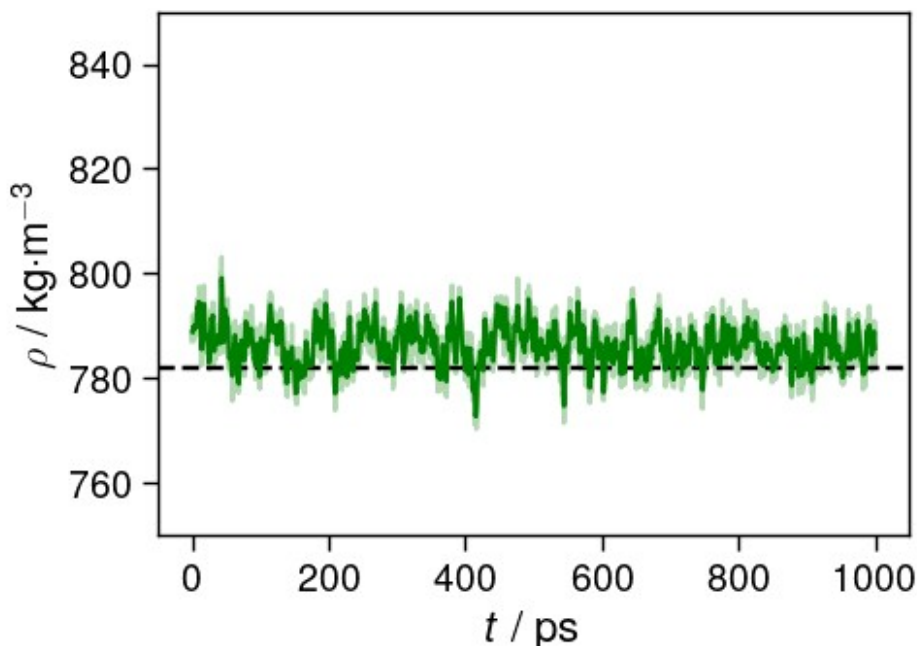
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(t, p, '-b', alpha=0.3)
ax.axhline(y=1, linestyle='--', color='k')
ax.plot(ma(t, 20), ma(p,20), '-b')
ax.set_ylabel(r'$p$ / bar')
ax.set_xlabel(r'$t$ / ps')
Text(0.5, 0, '$t$ / ps')
```



The pressure is generally allowed to fluctuate but the density should not.

```
data = np.loadtxt('MD/npt/density.xvg', skiprows=24)
t = data[:,0]
rho = data[:,1]
```

```
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(t, rho, '-g', alpha=0.3)
ax.axhline(y=782, linestyle='--', color='k')
ax.plot(ma(t, 20), ma(rho,20), '-g')
ax.set_ylim([750, 850])
ax.set_ylabel(r'\rho$ / kg$\cdot$m$^{-3}$')
ax.set_xlabel(r'$t$ / ps')
Text(0.5, 0, '$t$ / ps')
```



Production

Okay to run the final production run I simulate 10 ns in steps of 1 fs. **Also make sure that you again replace the compressibility in the prod.mdp file by that of your solvent**

I again ran:

```
gmx grompp -f prod.mdp -c npt/npt.gro -p topol.top -o prod/prod
```

on the cluster and used the following job script for the main calculation:

```
!cat MD/JOB_Prod.sh

#!/bin/bash
#SBATCH --job-name=Pe_DMA_NPT
#SBATCH --output=slurm-%J.out
#SBATCH --error=slurm-%J.err
#SBATCH --nodes=1
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=32
#SBATCH --time=00-4:00:00
#SBATCH --partition=shared-cpu

module load GCC/11.3.0 CUDA/11.4.1 OpenMPI/4.1.4 GROMACS/2023.1

gmx mdrun -nt 32 -v -deffnm prod/prod
```

Analyzing the Results

I ran:

```
gmx trjconv -s prod/prod.tpr -f prod/prod.xtc -o prod/prod-pe.xtc -pbc
mol -center
```

and centered the trajectory on the Pe molecule by selecting: Select group for centering = 2 (PER) and Select group for output = 0 (system). I also made a style file for VMD that we can use to look at the trajectory:

```
vmd -e style.sh
```

Let's analyze the `r_com` distance. For all these analyses we need index files (called `ndx` files). We make them using:

```
gmx make_ndx -f prod/prod.gro (q)
```

this will generate an `index.ndx` file. We will also make a folder called `analysis`.

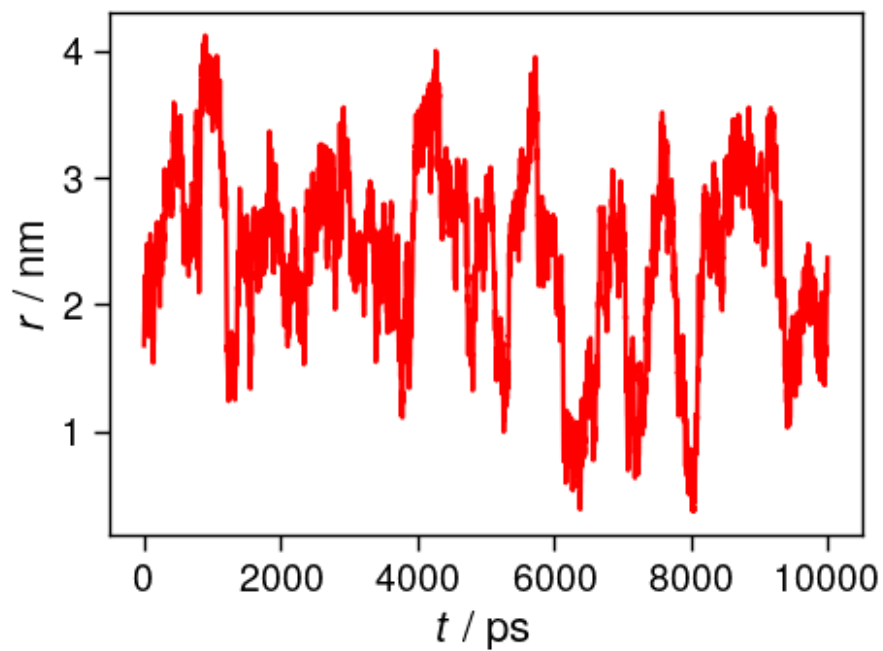
Center of Mass Distance

```
gmx distance -f prod/prod-pe.xtc -s prod/prod.tpr -n index.ndx -oav
analysis/com_dist.xvg
```

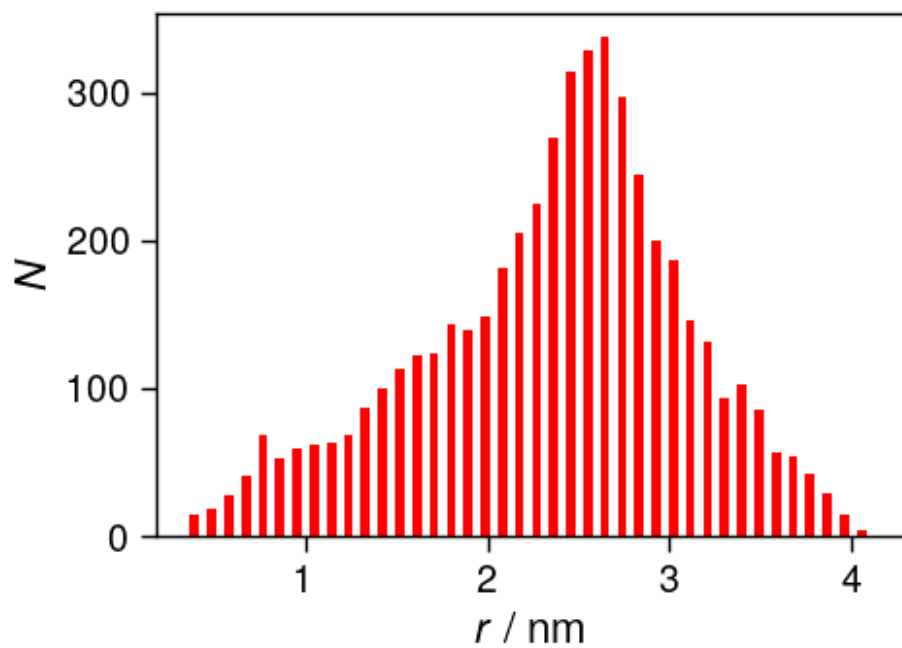
and then type `com of resname PER plus com of resname DMA` followed by `ctrl + D`

```
data = np.loadtxt('MD/analysis/com_dist.xvg', skiprows=24)
t = data[:,0]
r_com = data[:,1]

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(t, r_com, '-r')
ax.set_ylabel(r'$r$ / nm')
ax.set_xlabel(r'$t$ / ps')
Text(0.5, 0, '$t$ / ps')
```



```
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.hist(r_com, 40, color='r', width=0.05)
ax.set_ylabel(r'$N$')
ax.set_xlabel(r'$r$ / nm')
Text(0.5, 0, '$r$ / nm')
```



Minimum Distance

Essentially the same but the command is:

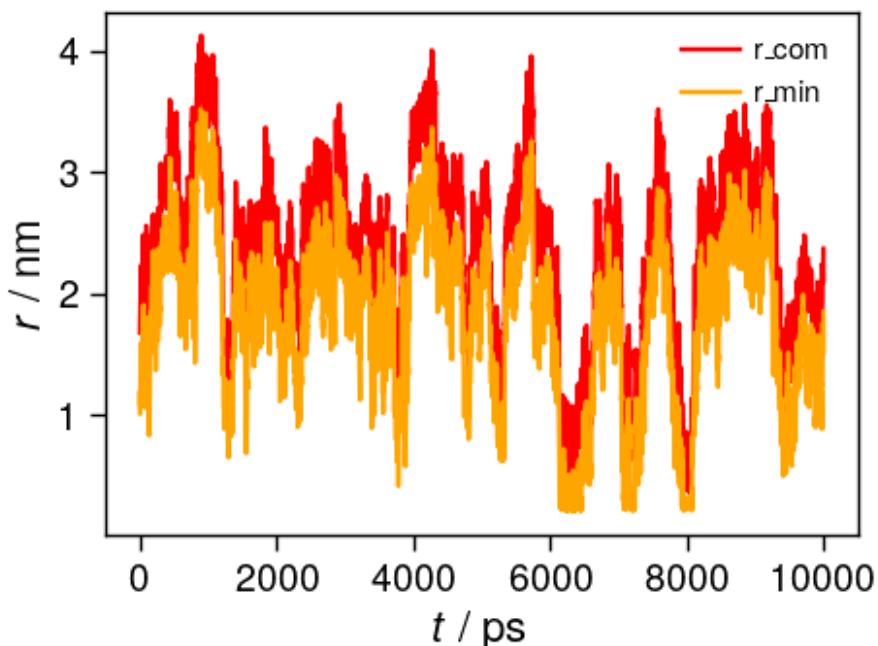
```
gmx mindist -f prod/prod-pe.xtc -s prod/prod.tpr -n index.ndx -od  
analysis/dee_dist.xvg
```

just select the groups after one another with enter in between.

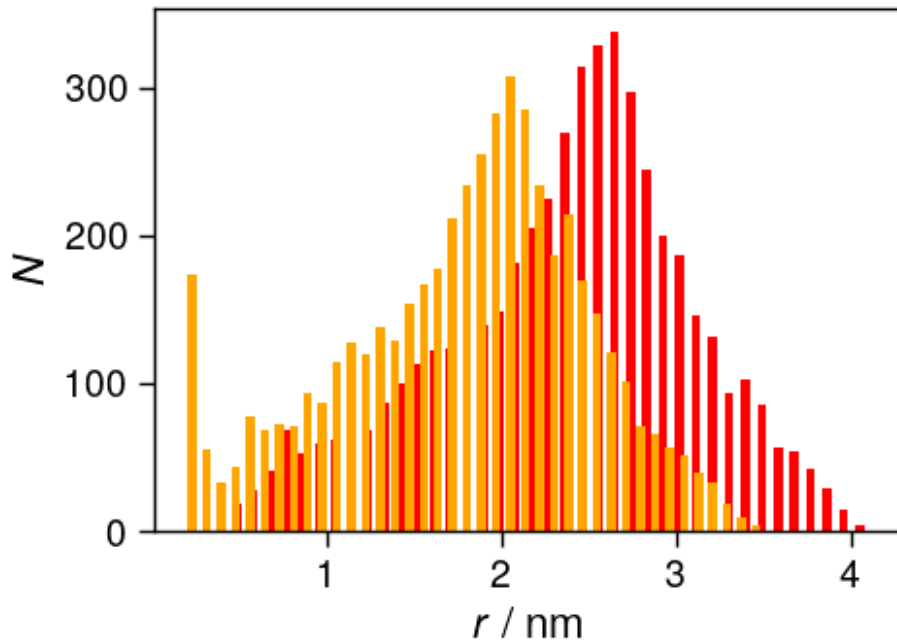
```
data = np.loadtxt('MD/analysis/dee_dist.xvg', skiprows=24)  
r_min = data[:,1]
```

```
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))  
ax.plot(t, r_com, '-r', label='r_com')  
ax.plot(t, r_min, '-o', color='orange', label='r_min')  
ax.set_ylabel(r'$r$ / nm')  
ax.set_xlabel(r'$t$ / ps')  
ax.legend()
```

<matplotlib.legend.Legend at 0x15536f2b0>



```
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))  
ax.hist(r_com, 40, color='r', width=0.05)  
ax.hist(r_min, 40, color='orange', width=0.05)  
ax.set_ylabel(r'$N$')  
ax.set_xlabel(r'$r$ / nm')  
Text(0.5, 0, '$r$ / nm')
```



Angle between Molecular Planes

Now we need to modify the index file to specify three atoms in each Pe and DMA to define the planes. We need to make an index for this:

```
gmx make_ndx -f prod/prod.gro
```

then take the atoms for the Peryle type: a 1 9 14 + ENTER and then for DMA: a 34 36 38 + ENTER then quit. Now two more groups of these planes are generated. To plot the angle of these two planes over time:

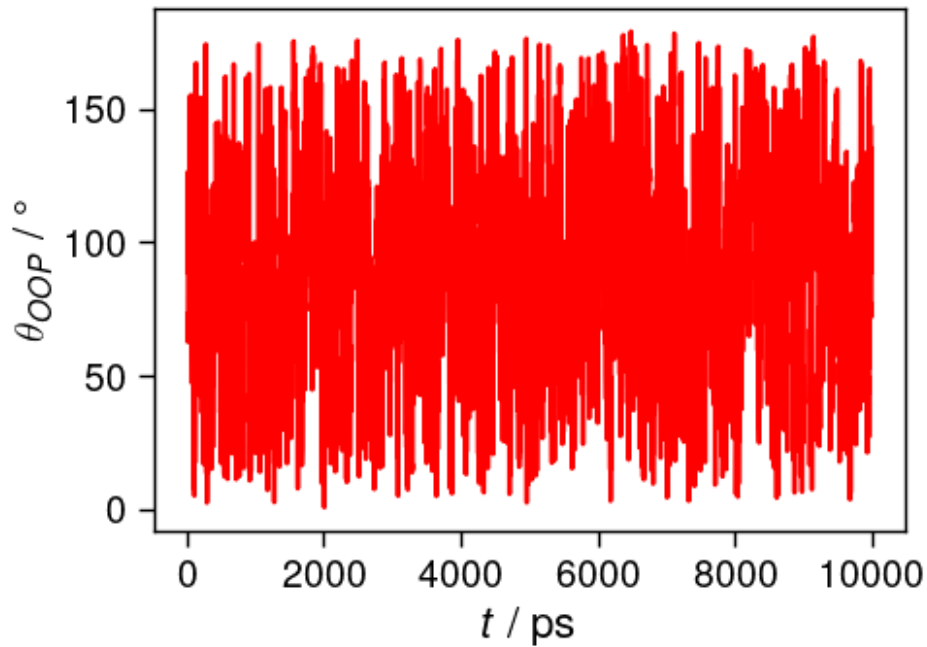
```
gmx gangle -f prod/prod-pe.xtc -s prod/prod.gro -g1 plane -g2 plane -n
index.ndx -oall analysis/plane_angle.xvg -group1 5 -group2 6
```

```
data = np.loadtxt('MD/analysis/plane_angle.xvg', skiprows=24)
t = data[:,0]
theta_OOP = data[:,1]
```

```
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(t, theta_OOP, '-r')
ax.set_ylabel(r'$\theta_{OOP}$ / °')
ax.set_xlabel(r'$t$ / ps')
```

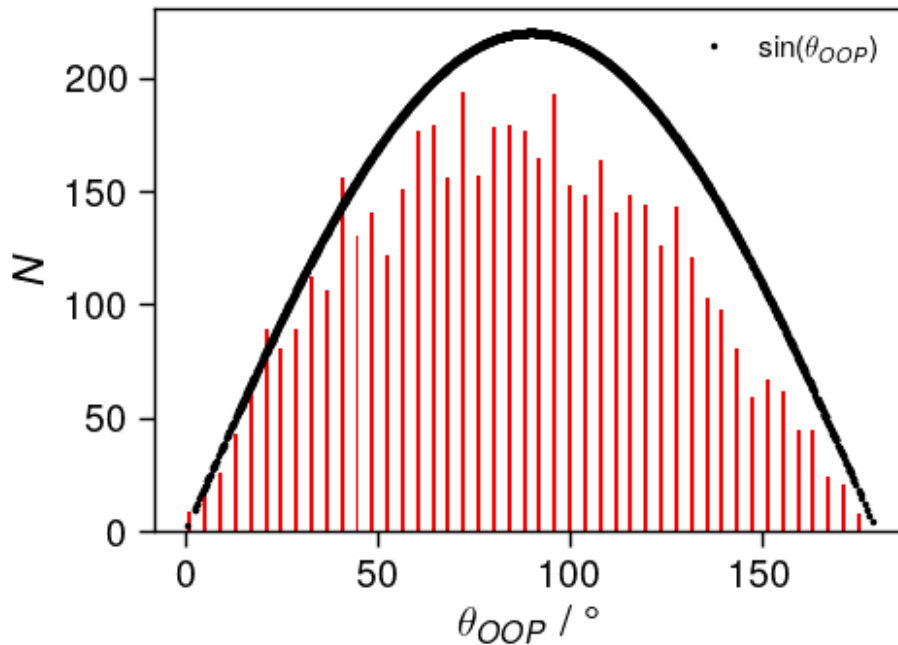
No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
<matplotlib.legend.Legend at 0x1555ed450>
```



```
fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.hist(theta_OOP, 45, color='r', width=1)
ax.plot(theta_OOP, np.sin(theta_OOP*np.pi/180)*220, '.k',
markersize=3, label=r'sin($\theta_{OOP}$)')
ax.set_ylabel(r'$N$')
ax.set_xlabel(r'$\theta_{OOP}$ / °')
ax.legend()

<matplotlib.legend.Legend at 0x1694bfb50>
```



Rotational Correlation Function

We need to again define two vectors or planes, but we can use the same planes as for the OOP angle. To generate the correlation function:

```
gmx rotacf -P 2 -f prod/prod-pe.xtc -s prod/prod.tpr -n index.ndx -o
analysis/rot_Pe.xvg
```

And we can choose group 5 for Pe or do the same again with group 6 for DMA.

```
data = np.loadtxt('MD/analysis/rot_Pe.xvg', skiprows=24)
t = data[:,0]
Pe = data[:,1]

data = np.loadtxt('MD/analysis/rot_DMA.xvg', skiprows=24)
t = data[:,0]
DMA = data[:,1]

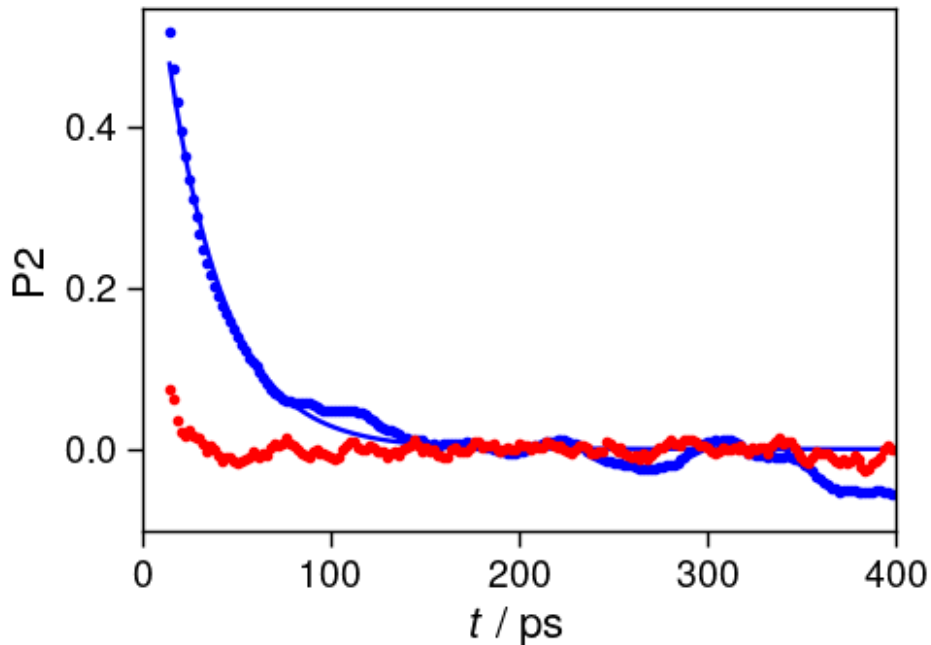
from VautheyLab.fit_functions import mono_exp

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))

p, pcov = curve_fit(mono_exp, t, Pe)

print(p)
ax.plot(t, mono_exp(t, *p), '-b')
ax.plot(t, Pe, '.b')
ax.plot(t, DMA, '.r')
ax.set_xlim([-0.1, 1.1])
ax.set_xlim([0, 400])
```

```
ax.set_ylabel(r'P2')
ax.set_xlabel(r'$t$ / ps')
[ 0.75675335 30.41992228]
Text(0.5, 0, '$t$ / ps')
```



Radial Distribution Function

We can again use the same index file. The commands is:

```
gmx rdf -f prod/prod-pe.xtc -s prod/prod.tpr -n index.ndx -o
analysis/Pe_rdf.xvg -bin 0.02
```

then select e.g. Pe and ACN.

```
%matplotlib inline

data = np.loadtxt('MD/analysis/Pe_rdf.xvg', skiprows=25)
r = data[:,0]
g_Pe = data[:,1]

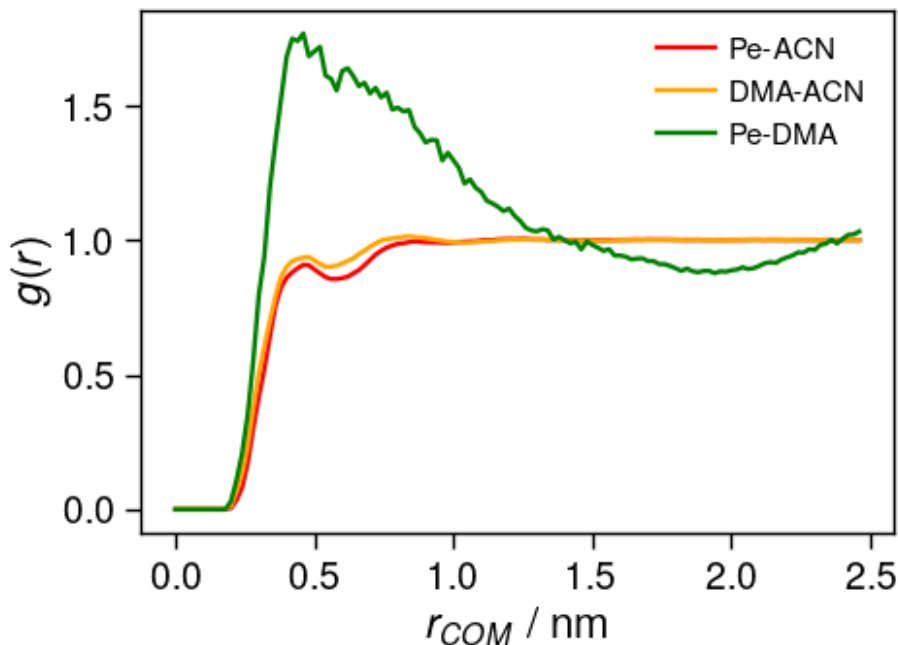
data = np.loadtxt('MD/analysis/DMA_rdf.xvg', skiprows=25)
r = data[:,0]
g_DMA = data[:,1]

data = np.loadtxt('MD/analysis/Pe_DMA_rdf.xvg', skiprows=25)
r = data[:,0]
g_Pe_DMA = data[:,1]
```

```

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(r, g_Pe, '-r', label='Pe-ACN')
ax.plot(r, g_DMA, '-', color='orange', label='DMA-ACN')
ax.plot(r, g_Pe_DMA, '-g', label='Pe-DMA')
ax.legend()
ax.set_ylabel(r'$g(r)$')
ax.set_xlabel(r'$r_{COM}$ / nm')
Text(0.5, 0, '$r_{COM}$ / nm')

```



Number of Solvent Molecules around a Solute

The command is:

```

gmx trjorder -f prod/prod-mol.xtc -s prod/prod.tpr -n index.ndx -na 6
-da 0 -r 0.5 -nshell analysis/nshell_Pe.xvg

```

where:

- na = number of atoms in the solvent
- da = if = 0 take com otherwise specify atom number
- r = shell thickness in nm

```

data = np.loadtxt('MD/analysis/nshell_Pe.xvg', skiprows=25)
t = data[:,0]
n_Pe = data[:,1]

data = np.loadtxt('MD/analysis/nshell_DMA.xvg', skiprows=25)

```

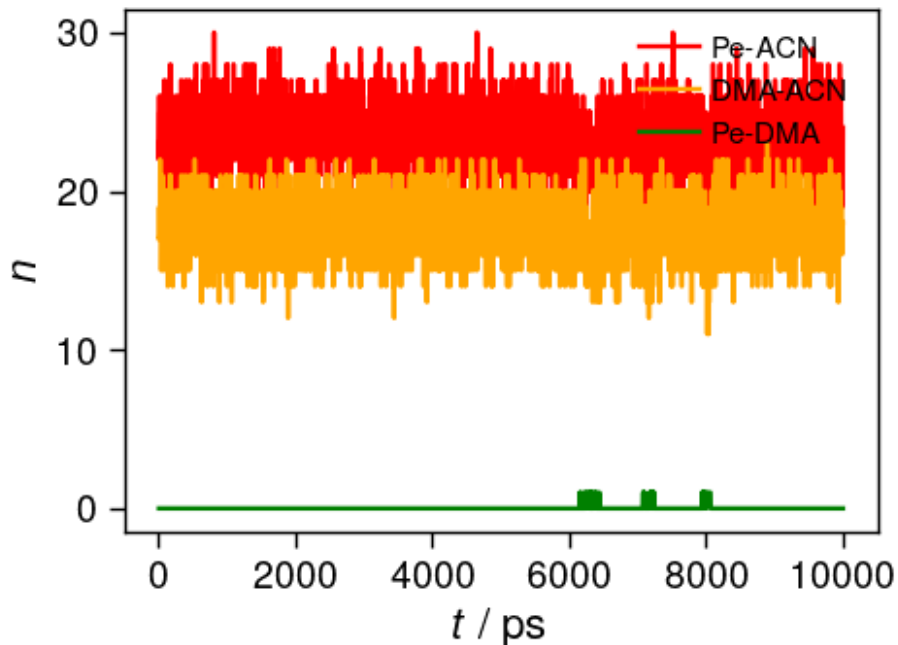
```

t = data[:,0]
n_DMA = data[:,1]

data = np.loadtxt('MD/analysis/nshell_Pe_DMA.xvg', skiprows=25)
t = data[:,0]
n_Pe_DMA = data[:,1]

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.plot(t, n_Pe, '-r', label='Pe-ACN')
ax.plot(t, n_DMA, '--', color='orange', label='DMA-ACN')
ax.plot(t, n_Pe_DMA, '-g', label='Pe-DMA')
#ax.set_ylim([-0.1, 1.1])
ax.legend()
ax.set_ylabel(r'$n$')
ax.set_xlabel(r'$t$ / ps')
Text(0.5, 0, '$t$ / ps')

```



```

fig, ax = plt.subplots(1,1,figsize=(5, 3.5))
ax.hist(n_Pe, 30, color='r', label='Pe-ACN', width=1)
ax.hist(n_DMA, 30, color='orange', label='DMA-ACN', width=1)
ax.hist(n_Pe_DMA, 30, color='g', label='Pe-DMA', width=1)
ax.legend()
ax.set_ylabel(r'$n$')
ax.set_xlabel(r'$n$')
Text(0.5, 0, '$n$')

```

